

Module 1: Lesson 1

# Multilayer Perceptrons (MLPs)



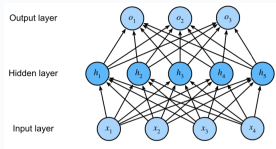
# Outline

- ▶ Revisiting MLPs
- ▶ Initialization: vanishing and exploding gradients
- ▶ A quick note on regularization
- ▶ Dropout

# Revisiting MLPs

During the last module of the Machine Learning course, we introduced a simple example of a deep Neural Network (NN): Multilayer Perceptrons (MLPs). This is precisely the point of departure for the Deep Learning for Finance course.

- Remember that we can express an MLP as a series of hidden layers with different units (neurons), each of which is activated (or not) given an activation function that incorporates the non-linearity into the process:



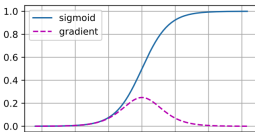
While we already know the basics about how these NNs work (e.g., forward and backprop), there are still many features to discuss that can help increase the efficiency and performance of our models. This is what we'll start doing in this lesson.

This lesson is based on Chapter 5 of Zhang et al.'s book *Dive into Deep Learning*.

# Parameter initialization

Let's start discussing these by looking at the first task we need to perform when training: parameter initialization.

- ▶ So far, we have given relatively low importance to the way parameters ( $W$  and  $b$ ) were initialized.
- ▶ We simply said that this was better done by selecting random numbers from a pre-specified distribution (e.g., uniform).
- ▶ However, the way that we initialize these parameters (e.g., the distribution we choose) can have significant effects on training.
- ▶ One of the most perilous effects of initialization choices is related to the (in)stability of gradients. The random choice of weights can lead to updates in parameters while training that are either:
  - Too large → Exploding gradient problem → difficult for gradient descent to converge
  - Too small → Vanishing gradient problem → difficult for algorithm to learn



# How to initialize parameters

Is there any way to get rid of problems related to exploding/vanishing gradients?

Sadly, there is no universal way to get rid of the problems related to parameter initialization. There are, however, some things that we can do in order to mitigate their impact:

► **Default initialization in Keras:** One lazy option is to trust that Keras is taking care of the problem for us:

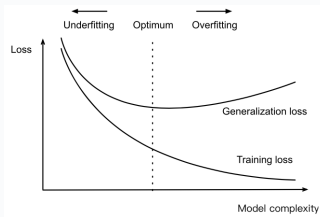
- Indeed, Keras has a default option for dense layers (the only ones we have seen so far) that relies on a uniform distribution called glorot uniform. Here, you have the link to the different default options when we set a dense layer: [https://keras.io/api/layers/core\\_layers/dense/](https://keras.io/api/layers/core_layers/dense/)
- Here, you have the link to the Keras documentation for all the different initializers: <https://keras.io/api/layers/initializers/>

► **Xavier initialization:**

- Probably the one most commonly used in practice (actually glorot uniform is a type of Xavier)
- The typical Xavier initialization uses random weights from a Gaussian (or uniform) distribution with zero mean but conditioning the variance to avoid exploding/vanishing problems.

# Regularization

Machine learning models, as we've already learned, are always subject to the trade-off between in- and out-of-sample performance. Deep learning models based on neural networks are not an exception to this problem:



When our models did overfit, we already worked out some solutions based on **regularization** of the loss function. These are also applicable in deep learning models, as we will see:

- ▶ Lasso ( $\ell_1$ -norm)
- ▶ Ridge ( $\ell_2$ -norm)
- ▶ Early Stopping

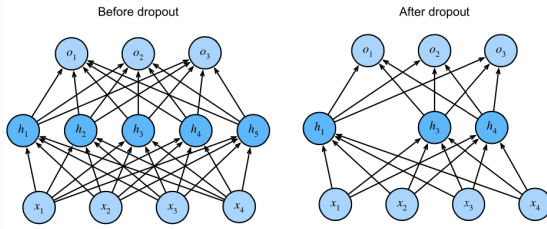
# Dropout

A common form of regularization in deep learning that we have not yet talked about is **Dropout**.

Regularization norms are, in essence, ways to aim for a simpler model, which makes sense in light of the previous graph.

Dropout is a little different because it basically consists of adding noise to the model:

- ▶ It is called **dropout** because we literally drop out some neurons during model training.
- ▶ On each iteration during model training, we will zero-out (de-activate) a fraction of units in each layer.
- ▶ Typically, we will not use dropout in the test sample.



# Summary of Lesson 1

In Lesson 1, we have:

- ▶ Revisited MLPs
- ▶ Discussed different options for parameter initialization
- ▶ Introduced dropout

⇒ **References for this lesson:**

Chapter 5, Sections 5.4 to 5.7 of [Zhang, Ashton, et al. \*Dive into Deep Learning\*](#).

**TO DO NEXT:** This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will implement all these novel things in practice in an increasingly deeper neural network based on MLPs.